

PROJETO 7

ORGANIZANDO O CÓDIGO COM FUNÇÕES

Evolução da porta automática do Projeto 6

OBJETIVO

Reorganizar um programa que já funciona usando funções criadas por nós. Nesta aula não acrescentaremos sensores, atuadores, fórmulas ou estruturas novas ao projeto. O conteúdo novo é a criação, a chamada e o retorno de funções.

1. O problema que já sabemos resolver

No Projeto 6, a porta automática já mede a distância com o HC-SR04, transforma a distância em proximidade, abre dois servos em movimentos espelhados, controla LEDs e produz aviso sonoro. O programa funciona, mas o loop() ficou responsável por muitas tarefas.

```
void loop() {  
  // medir distância  
  // calcular proximidade  
  // mostrar no Serial  
  // abrir ou fechar a porta  
  // controlar LEDs  
  // controlar buzzer  
}
```

PERGUNTA DA AULA

Se algumas partes do programa realizam tarefas bem definidas, podemos dar um nome a cada tarefa e chamá-la quando necessário?

2. Você já usa funções

Desde os primeiros programas usamos setup() e loop(). Eles são funções. Também chamamos funções prontas, como pinMode(), digitalWrite(), delay(), tone(), analogRead() e Serial.print(). Até agora, porém, não havíamos criado nossas próprias funções.

```
void setup() {  
  // executado uma vez  
}  
  
void loop() {  
  // executado repetidamente  
}
```

3. Primeira função criada por nós

Vamos começar pelo caso mais simples: uma função que executa comandos e não devolve um valor.

```
void som() {  
  
    tone(BUZZER, 1200);  
    delay(100);  
  
    noTone(BUZZER);  
    delay(300);  
}
```

Leitura da estrutura:

void	som	()	{ ... }
não devolve valor	nome da função	sem parâmetros nesta aula	comandos executados

4. Criar não é executar

Escrever a função apenas define o que ela fará. Para executá-la, fazemos uma chamada:

```
som();
```

Quando o programa encontra som();, ele vai até a função som(), executa seus comandos e depois retorna ao ponto seguinte.

5. Função para abrir a porta

```
void abrirPorta() {  
  
    digitalWrite(LED_VERMELHO, LOW);  
    digitalWrite(LED_VERDE, HIGH);  
  
    if (aberta == 0) {  
  
        for (int angulo = 0; angulo <= 90; angulo++) {  
  
            servo1.write(angulo);  
            servo2.write(180 - angulo);  
  
            delay(10);  
        }  
  
        aberta = 1;  
    }  
}
```

Nada novo foi acrescentado à lógica: são os mesmos digitalWrite(), if, for, write() e delay() do Projeto 6. A diferença é que esse bloco ganhou um nome.

6. Função para fechar a porta

```
void fecharPorta() {  
    noTone(BUZZER);  
  
    if (aberta == 1) {  
        for (int angulo = 90; angulo >= 0; angulo--) {  
            servo1.write(angulo);  
            servo2.write(180 - angulo);  
  
            delay(10);  
        }  
  
        aberta = 0;  
    }  
  
    digitalWrite(LED_VERMELHO, HIGH);  
    digitalWrite(LED_VERDE, LOW);  
}
```

7. Uma função que devolve um resultado

A medição é diferente. Não queremos apenas executar comandos: precisamos obter a distância para continuar o programa. Por isso a função devolverá um valor do tipo float.

```
float medirDistancia() {  
  
    digitalWrite(TRIG, LOW);  
    delayMicroseconds(2);  
  
    digitalWrite(TRIG, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(TRIG, LOW);  
  
    unsigned long duracao = pulseIn(ECHO, HIGH);  
  
    float distancia = duracao / 58.0;  
  
    return distancia;  
}
```

RETURN

return envia um valor de volta para o ponto em que a função foi chamada. Aqui, medirDistancia() devolve a distância calculada.

8. Recebendo o valor retornado

```
float distancia = medirDistancia();
```

Primeiro medirDistancia() é executada. O valor indicado em return distancia; volta para o loop() e é armazenado na variável distancia.

9. O loop() começa a mostrar a lógica principal

```
void loop() {  
  
    float distancia = medirDistancia();  
  
    int proximidade = map(distancia, 400, 0, 0, 100);  
    proximidade = constrain(proximidade, 0, 100);  
  
    if (proximidade >= 70) {  
  
        abrirPorta();  
        som();  
    }  
  
    else {  
  
        fecharPorta();  
        delay(200);  
    }  
}
```

Agora o `loop()` permite enxergar rapidamente a decisão principal: medir, calcular a proximidade e decidir entre abrir ou fechar.

10. Antes × depois

ANTES	DEPOIS
Grande parte das tarefas escrita diretamente dentro do <code>loop()</code> .	Tarefas com nomes: <code>medirDistancia()</code> , <code>abrirPorta()</code> , <code>fecharPorta()</code> e <code>som()</code> .
É necessário ler vários comandos para descobrir o que cada trecho faz.	O nome da função ajuda a indicar a responsabilidade daquele bloco.
Alterações exigem localizar o trecho dentro de um bloco maior.	Cada tarefa fica concentrada em uma parte específica do programa.

11. Fluxo do programa organizado

```
setup()
|
v
loop()
|
+--> medirDistancia()
|
+--> map() + constrain()
|
+--> if proximidade >= 70
|   |
|   +--> abrirPorta()
|   +--> som()
|   else
|       |
|       +--> fecharPorta()
```

12. Regras desta primeira aula de funções

- Vamos criar funções sem parâmetros: usamos `()` vazio.
- Funções void executam uma tarefa e não devolvem um valor.
- A função float `medirDistancia()` devolve um número com casas decimais.
- `return` é usado para devolver o resultado da função.
- A função precisa ser chamada para executar.
- Não vamos trabalhar parâmetros nesta aula.

13. Prática guiada — refatorando o Projeto 6

Abra uma cópia do Projeto 6. Não altere o `diagram.json`, o `wokwi.toml`, o `platformio.ini` nem as ligações. A mudança será somente na organização do `main.cpp`.

1. Compile e execute primeiro a versão original para confirmar que a porta funciona.

2. Crie a função `som()` e substitua o bloco do buzzer por `som()`;
3. Crie `abrirPorta()` e substitua o bloco de abertura por `abrirPorta()`;
4. Crie `fecharPorta()` e substitua o bloco de fechamento por `fecharPorta()`;
5. Crie `medirDistancia()` com retorno float e use `float distancia = medirDistancia()`;
6. Compile novamente e teste as mesmas distâncias usadas no Projeto 6.
7. Compare o comportamento antes e depois. O funcionamento deve permanecer equivalente.

14. Teste orientado

Distância	Proximidade	Resultado esperado	Observado
400 cm	0%	Porta fechada	
200 cm	50%	Porta fechada	
120 cm	70%	Porta aberta + aviso	
80 cm	80%	Porta aberta + aviso	

15. Exercício

Sem consultar o código completo, reorganize sua cópia do Projeto 6 utilizando exatamente estas quatro funções:

```
void som()
void abrirPorta()
void fecharPorta()
float medirDistancia()
```

Depois responda:

1. Qual das quatro funções usa `return`?

2. Por que `medirDistancia()` começa com `float` e `som()` começa com `void`?

3. O que acontece quando o programa encontra `abrirPorta()`?

4. O comportamento do circuito mudou depois da reorganização? Explique.

5. Qual versão ficou mais fácil de localizar a lógica de abertura e fechamento?

NÃO É CONTEÚDO DESTA AULA

Funções com parâmetros serão estudadas depois. Nesta aula, mantenha os parênteses vazios nas funções criadas.

16. Código consolidado do Projeto 7

Depois de construir e compreender cada função separadamente, reúna todas as partes. A versão completa do main.cpp fica assim:

```
#include <Arduino.h>
#include <Servo.h>

const int TRIG = 11;
const int ECHO = 10;
const int LED_VERMELHO = 8;
const int LED_VERDE = 9;
const int BUZZER = 12;

Servo servo1;
Servo servo2;

int aberta = 0;

void som() {

    tone(BUZZER, 1200);
    delay(100);

    noTone(BUZZER);
    delay(300);
}

void abrirPorta() {

    digitalWrite(LED_VERMELHO, LOW);
    digitalWrite(LED_VERDE, HIGH);

    if (aberta == 0) {

        for (int angulo = 0; angulo <= 90; angulo++) {

            servo1.write(angulo);
            servo2.write(180 - angulo);

            delay(10);
        }

        aberta = 1;
    }
}

void fecharPorta() {

    noTone(BUZZER);

    if (aberta == 1) {

        for (int angulo = 90; angulo >= 0; angulo--) {

            servo1.write(angulo);
            servo2.write(180 - angulo);

            delay(10);
        }

        aberta = 0;
    }

    digitalWrite(LED_VERMELHO, HIGH);
    digitalWrite(LED_VERDE, LOW);
}

float medirDistancia() {

    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);

    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
```

```
digitalWrite(TRIG, LOW);

unsigned long duracao = pulseIn(ECHO, HIGH);

float distancia = duracao / 58.0;

return distancia;
}

void setup() {

  Serial.begin(9600);

  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);

  pinMode(LED_VERMELHO, OUTPUT);
  pinMode(LED_VERDE, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  servo1.attach(5);
  servo2.attach(6);

  servo1.write(0);
  servo2.write(180);

  digitalWrite(LED_VERMELHO, HIGH);
  digitalWrite(LED_VERDE, LOW);

  noTone(BUZZER);
}

void loop() {

  float distancia = medirDistancia();

  int proximidade = map(distancia, 400, 0, 0, 100);
  proximidade = constrain(proximidade, 0, 100);

  Serial.print("Distancia: ");
  Serial.print(distancia, 1);
  Serial.print(" cm | Proximidade: ");
  Serial.print(proximidade);
  Serial.println("");

  if (proximidade >= 70) {

    abrirPorta();
    som();
  }

  else {

    fecharPorta();
    delay(200);
  }
}
```

Observe que o funcionamento continua sendo o mesmo do Projeto 6. A diferença é a organização: cada tarefa principal agora possui uma função com nome próprio.